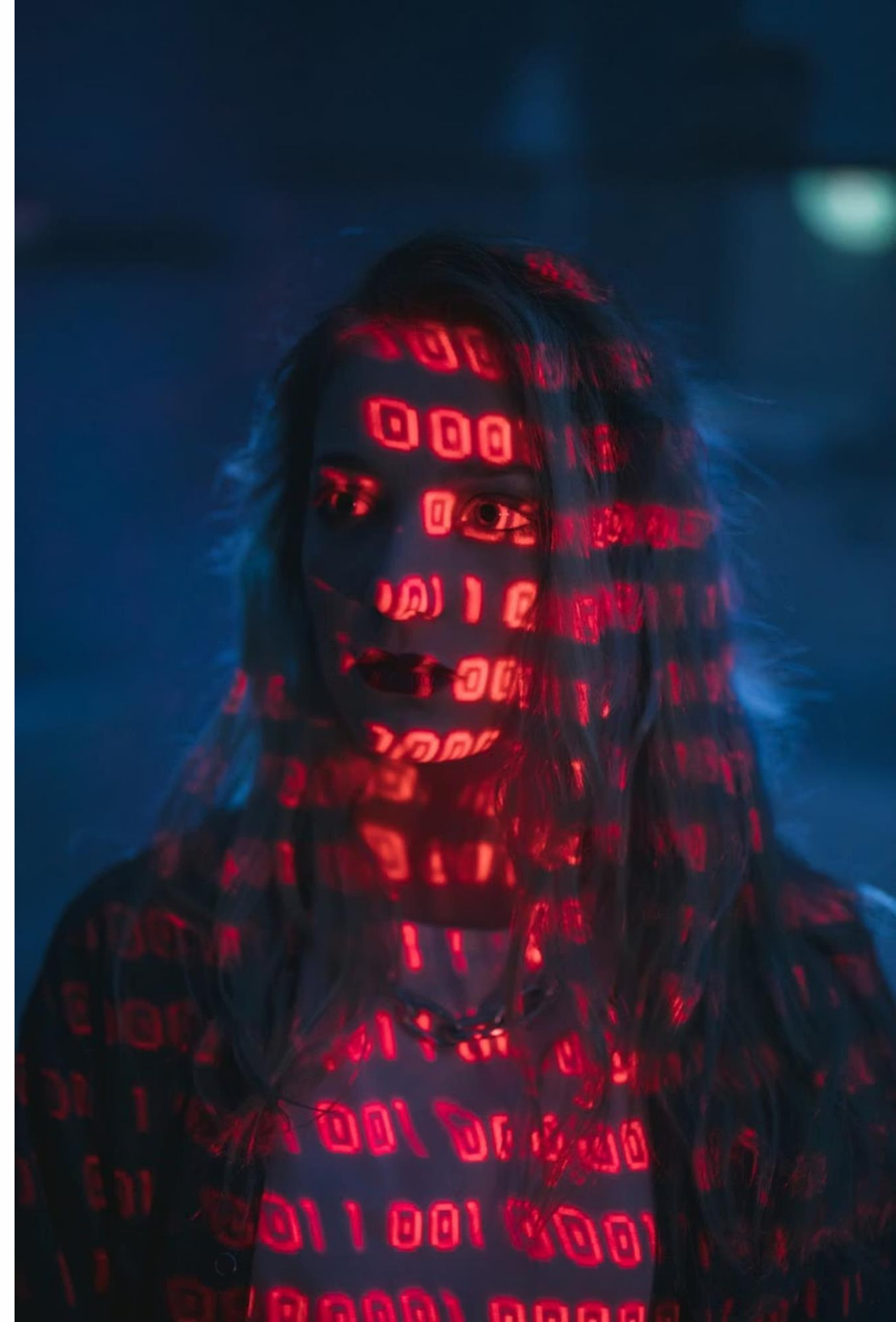


Cryptographic Signing and Verification of Terraform Plans for Supply Chain Integrity

Bridging the Gap Between Developer Intent and Infrastructure Reality

J.R.M.Sulakshana B.Karunarathne

February 2026



Agenda

1. Problem & Research Gap
2. Stakeholder Analysis
3. Requirements & Use Cases
4. System Architecture
5. Security Pipeline (4 Layers)
6. Timeline & Progress
7. Live Demonstration
8. Challenges & Solutions
9. Future Work & Conclusion

The Critical Challenge: Securing Infrastructure as Code

Infrastructure as Code (IaC) offers unparalleled agility, but with great power comes significant risk. A single misconfiguration or compromised credential can lead to catastrophic production infrastructure breaches, data loss, or service disruption.

Traditional security tools often operate in silos, creating a fragmented and reactive defense posture. They lack the unified visibility and cryptographic assurances needed to verify the integrity and provenance of IaC deployments.

The core gap remains: no single framework exists that cryptographically verifies **who** authored the code and **who** approved its deployment, leaving critical vulnerabilities in the software supply chain.



Problem Definition & Research Gap

The Problem

- Infrastructure as Code (IaC) introduces new attack vectors
- Misconfigurations lead to critical vulnerabilities
- Lack of visibility into IaC changes and deployments

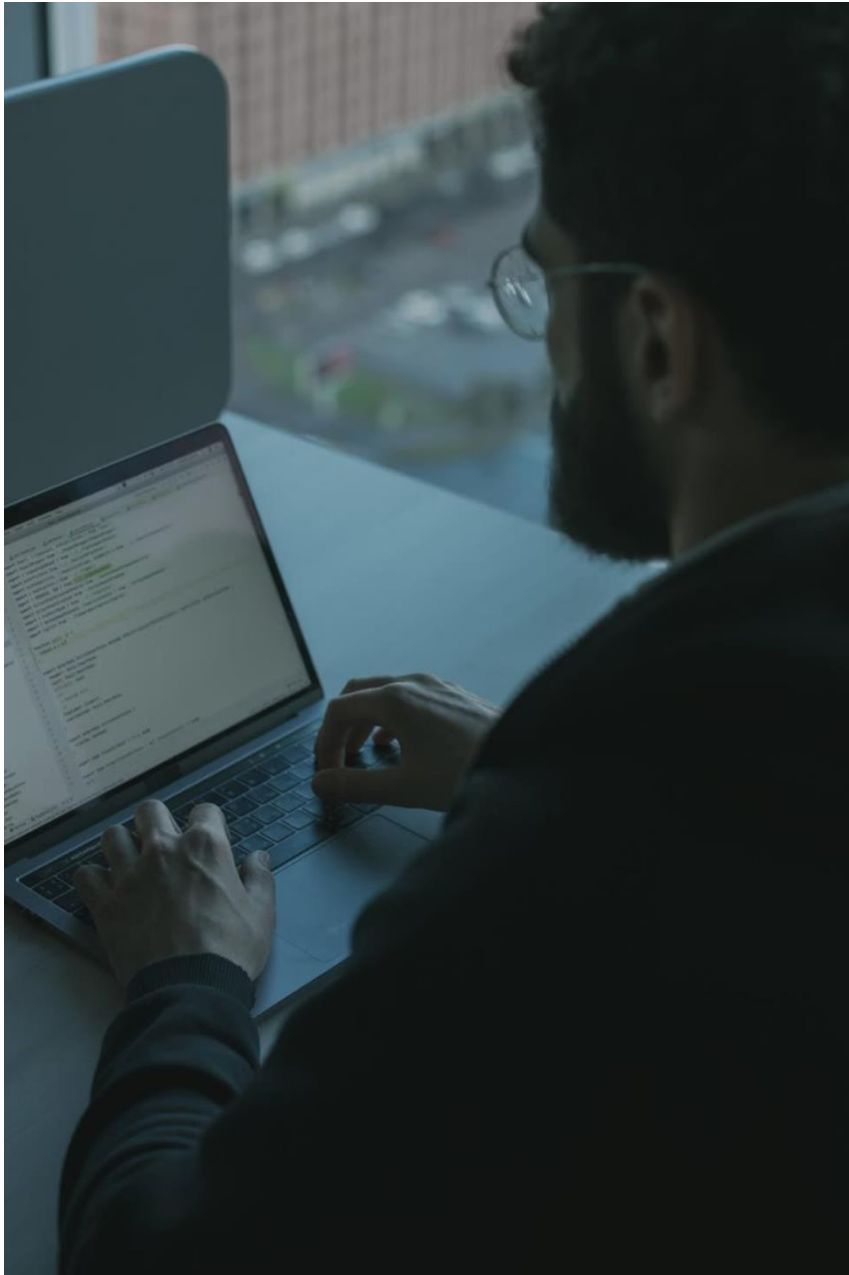
Research Gaps

- No unified chain of custody (NIST SP 800-218, 2022)
- Separation of duties not enforced (IEEE Security & Privacy, 2021)
- No emergency response mechanisms (SANS Institute, 2023)

Why a Security Framework?

- Ensure consistent security posture across all IaC
- Achieve compliance with industry regulations and standards
- Enable proactive identification and mitigation of risks

Layer 1: Identity & Integrity - Proving Authorship



This foundational layer prevents impersonation and ensures that every line of IaC can be traced back to its legitimate author. It creates an undeniable link between developer and code.

- **Technology:** Git GPG Signing
- **Mechanism:** Pre-push hooks
- **Key Achievement:** Our system automatically rejects unsigned commits at the Git push level. This proactive measure ensures that no anonymous or unverified code ever enters the repository, establishing a trusted baseline for all infrastructure changes.

Layer 2: Approval & Non-Repudiation - The Two-Person Rule

Enforcing separation of duties, this layer prevents unauthorized deployments even if a developer's credentials are compromised. It mandates explicit administrative approval for every infrastructure change.

- **Technology:** Cosign, Digital Signatures
- **Workflow:**
 - a. CI generates a Terraform plan.
 - b. Admin reviews the plan and signs it digitally.
 - c. CI pipeline validates the signature's authenticity and integrity.
 - d. Terraform apply proceeds only upon valid signature.
- **Key Achievement:** CI pipelines are hard-coded to refuse deployment without a valid, cryptographically verifiable admin signature, embodying a critical "two-person rule" for infrastructure changes.



Layers 3 & 4: Compliance & Provenance - Automated Guardians

These layers introduce automated guardrails and an immutable record, ensuring all infrastructure changes adhere to defined policies and can be fully audited post-deployment.



- **Compliance (Layer 3):**
 - **Technology:** Open Policy Agent (OPA) with Rego policies.
 - **Mechanism:** Policies are run *before*
 - **Key Achievement:** If an IaC change violates a pre-defined policy (e.g., attempting to provision a "Public S3 Bucket"), it is automatically flagged and prevented from being signed or deployed. This integrates security checks early in the pipeline.
- **Provenance (Layer 4):**
 - **Technology:** SLSA standards.
 - **Mechanism:** Generates tamper-proof provenance files.
 - **Key Achievement:** Every step of the IaC lifecycle, from commit to deployment, is meticulously recorded in an auditable, tamper-resistant log, offering a complete chain of custody for all infrastructure changes.

Novel Feature: Emergency Lockdown Mode

Addressing critical operational security scenarios, TerraSign introduces a "kill switch" to immediately halt all deployments in the event of an emergency, such as a compromised administrative key or a zero-day exploit.

The Problem: Compromised Admin Key

Even with multi-layer security, what if an administrative signing key is stolen? The risk of malicious or unauthorized deployments remains.

The Solution: ts-lockdown

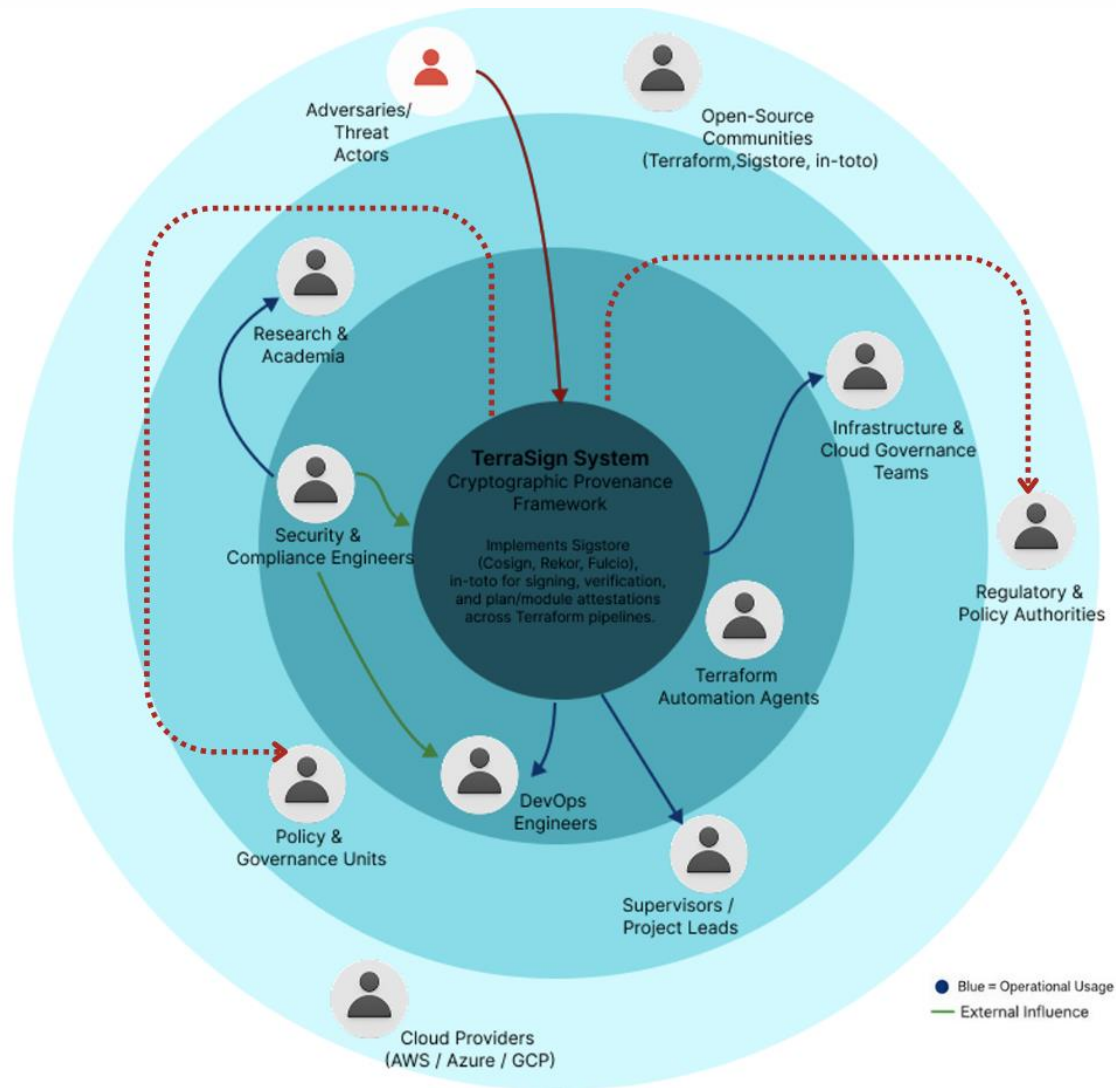
A server-side "Kill Switch" that instantly stops ALL infrastructure deployments across all environments. It acts as the ultimate failsafe.

Secure Restoration

Reactivating deployments (`ts-lockdown off`) requires multi-factor, strong authentication, preventing easy re-enablement by a single attacker.



Stakeholder Analysis



Key Stakeholder Requirements

Developers

- Fast signing
- Clear error messages

Admins

- Interactive dashboard
- Plan inspection

Security

- Policy enforcement
- Lockdown mode

Regulators

- Immutable audit logs (SLSA provenance)

Functional Requirements (SRS)

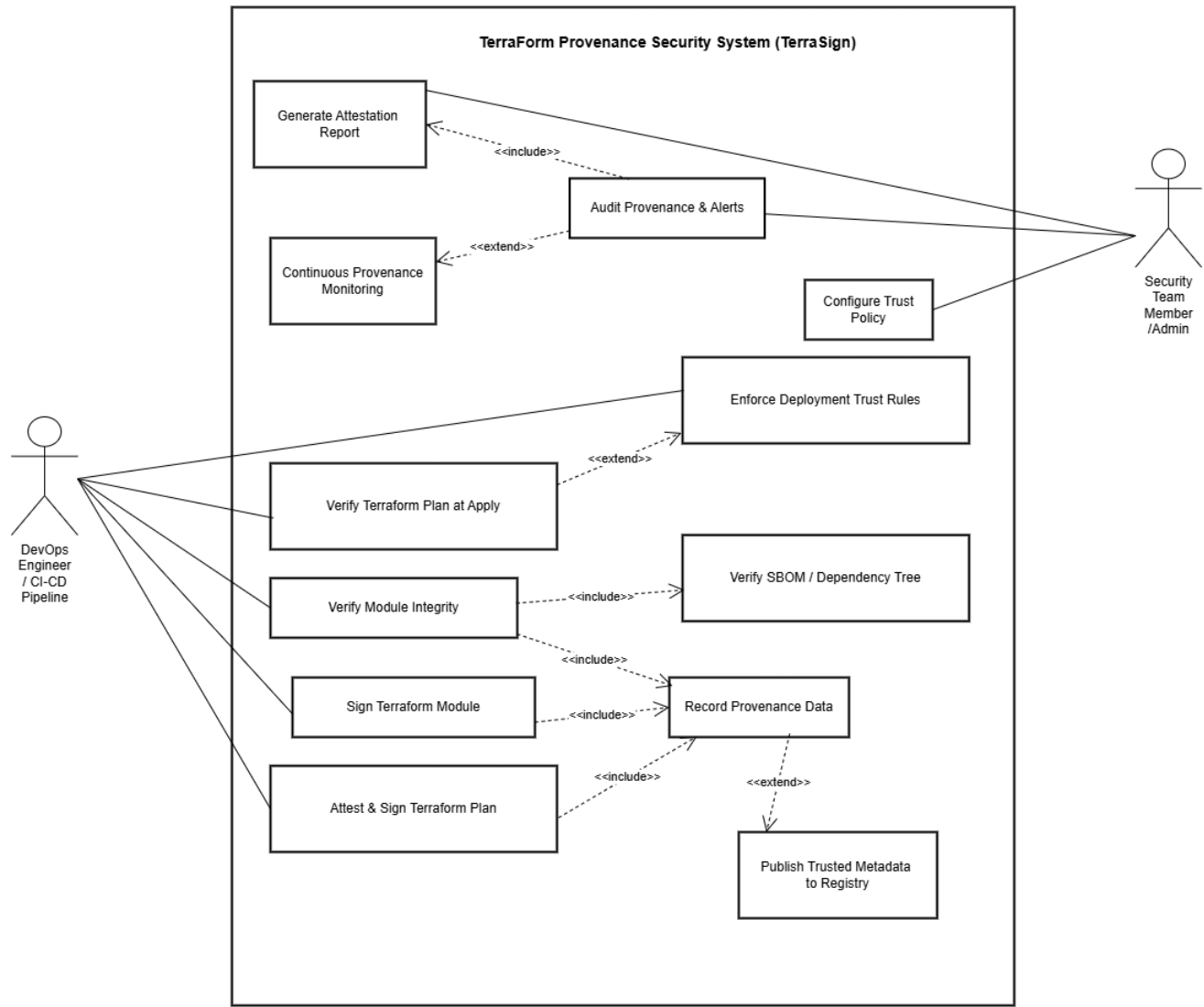
ID	Requirement	Priority (MoSCoW)	Status
FR1	Sign Terraform plans with Cosign	Must Have	✅ Implemented
FR2	Verify signatures before apply	Must Have	✅ Implemented
FR3	Submit plans to remote server	Must Have	✅ Implemented
FR4	Admin approval workflow	Must Have	✅ Implemented
FR5	Policy enforcement (OPA)	Must Have	✅ Implemented
FR6	SLSA provenance generation	Should Have	✅ Implemented
FR7	Git commit signing (GPG)	Should Have	✅ Implemented
FR8	Emergency lockdown mode	Should Have	✅ Implemented
FR9	Interactive monitoring dashboard	Could Have	✅ Implemented
FR10	Multi-party approval (M-of-N)	Won't Have	❌ Pending

Non-Functional Requirements

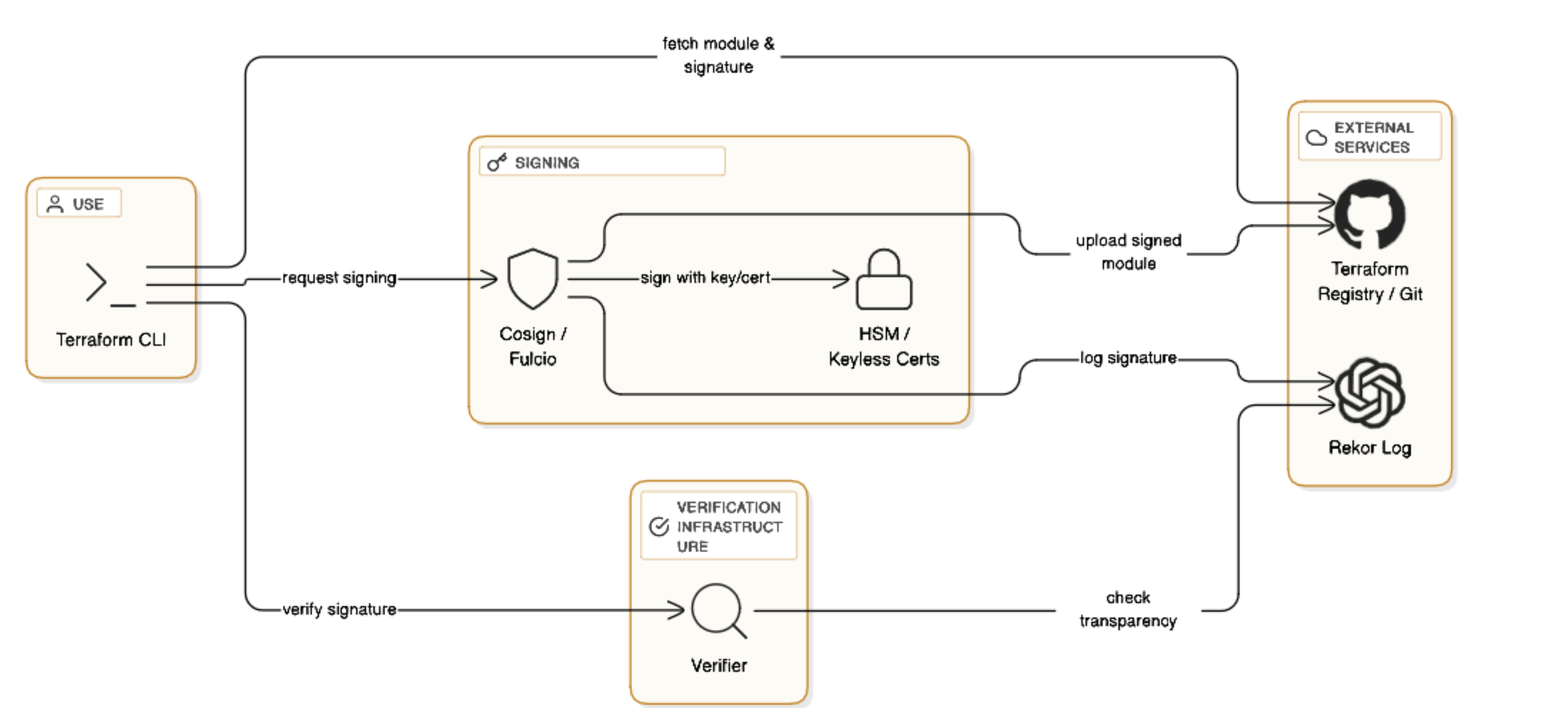
ID	Requirement	Target	Status
NFR1	Signature verification time	< 2 seconds	✓ Achieved
NFR2	Plan storage encryption	AES-256	✓ Implemented
NFR3	CLI usability	Single command	✓ Achieved
NFR4	CI/CD integration	Jenkins, GitHub Actions	✓ Implemented
NFR5	Auditability	Immutable logs	✓ SLSA provenance
NFR6	Security	No plaintext secrets	✓ Cosign keyless

Use Case Diagram & Traceability

This diagram illustrates the core interactions within TerraSign, mapping actors to their specific use cases.



System Architecture (Tier Architecture)



Development Timeline & Progress



Deviations & Achievements

- Added Emergency Lockdown (unplanned feature)
- Added Interactive Monitor (unplanned feature)
- Delayed: Multi party approval (moved to future work)

Milestones, Challenges & Solutions

Milestone	Challenge	Solution	Evidence
Signature Integration	Detached signatures not portable	Implemented bundle format	signer.go
CI/CD Integration	Jenkins version mismatch	Manual Terraform upgrade	Jenkinsfile
Policy Engine	OPA integration complexity	Embedded Rego engine	engine.go
Commit Signing	Git hooks not interactive	exec < /dev/tty redirection	pre-push hook
User Experience	CLI too verbose	Created ts-monitor dashboard	monitor.go
Security	Unsigned commits bypassing	Interactive menu with auto-sign	pre-push hook

Future Work & Remaining Tasks

By March 31st (Remaining Work)

- Complete documentation
- Final end to end testing
- Performance benchmarks
- Security audit

Future Improvements (Post-Project)

- Multi-party Approval (M-of-N signatures) - Require 2 out of 3 admins to approve critical changes
- Hardware Security Module (HSM) Support - Store signing keys in tamper-proof hardware
- Web UI Dashboard - Replace terminal UI with web interface
- Slack/Teams Integration - Notify admins of pending approvals

Conclusion

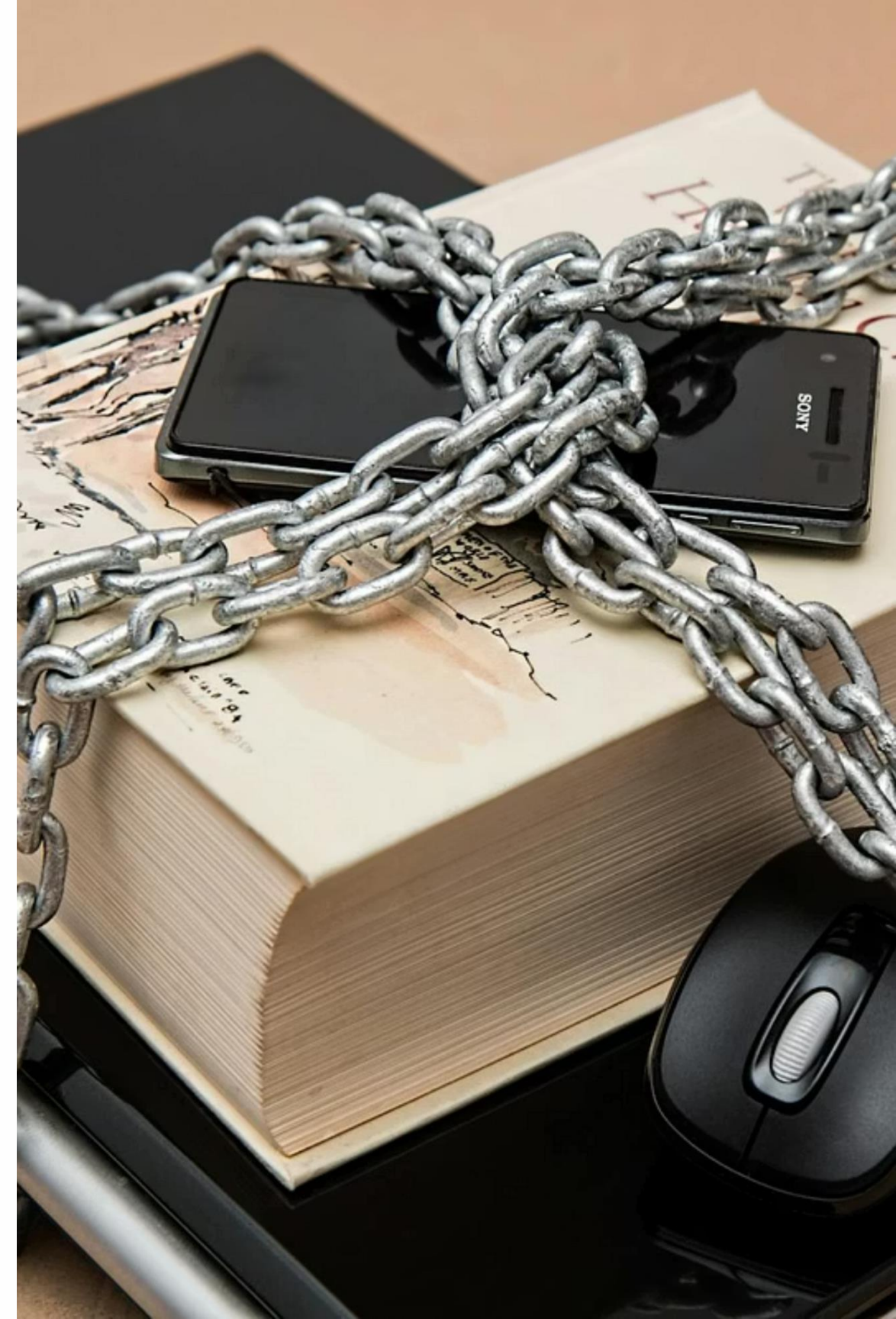
Summary

- TerraSign provides a complete Chain of Custody for infrastructure changes
- 4-layer defense prevents unauthorized, malicious, and accidental deployments
- Cryptographic proof of who wrote code and who approved it
- Emergency response capability for security incidents

Research Contribution

- First unified framework combining commit signing, plan signing, policy enforcement, and provenance for IaC
- Demonstrates "defense in depth" for infrastructure security

"TerraSign ensures that what you intend is exactly what you deploy - with proof."



Thank you for your attention

